

1. Software Test Guide

CLARA

Document Control	
Author(s)	Michael Gardner
Version	0.1.0
Status	Draft
Status Date	2025-12-29
SPDX-License-Identifier	BSD-3-Clause
License File	See the LICENSE file in the project root
Copyright	© 2025 Michael Gardner, A Bit of Help, Inc.

Project Profile	
Variant	library
Library Role	utility
Application Role	N/A
Assurance	spark-targeted
Execution	sequential
Parallelism	none
Platform	desktop, server, embedded
Execution Environment	linux, macos, windows, ada-runtime
Processor Architecture	amd64, arm64, arm32
Deployment	native

Table of Contents

1. Software Test Guide	1
2. Introduction	1
2.1. Purpose	1
2.2. Scope	1
2.3. References	1
3. Test Strategy	1
3.1. Test Categories	1
3.2. Testing Philosophy	1
3.3. Coverage Requirements	1
4. Test Organization	1
4.1. Directory Structure	1
4.2. Build/Test Projects	2
5. Test Execution	2
5.1. Running All Tests	2
5.2. Running Specific Suites	2
6. Test Details	2
6.1. Unit Tests	2
6.1.1. Flag Tests	2
6.1.2. Option Tests	2
6.1.3. Positional Tests	3
6.1.4. Parse Error Tests	3
6.1.5. Help Generation Tests	3
6.2. Integration Tests	3
6.2.1. Full Parse Scenarios	3
6.2.2. Edge Cases	3
6.3. SPARK Verification	4
6.3.1. Packages Under SPARK	4
6.3.2. Proof Objectives	4
7. Traceability	4
7.1. Requirements to Tests	4
8. Test Maintenance	4
8.1. When to Update	4
8.2. Quality Guidelines	4
8.3. Continuous Integration	5
9. Appendices	6
9.1. Change History	6

2. Introduction

2.1. Purpose

This Software Test Guide (STG) defines the testing strategy, test cases, and verification approach for **CLARA** (Command Line Arguments for Reliable Applications), a type-safe command-line argument parsing library for Ada 2022.

2.2. Scope

This document covers:

- test strategy and approach,
- unit test specifications,
- integration test specifications,
- SPARK verification,
- coverage requirements, and
- traceability.

2.3. References

- Software Requirements Specification (SRS).
- Software Design Specification (SDS).

3. Test Strategy

3.1. Test Categories

Category	Location	Purpose	Target
Unit Tests	test/unit/	Verify individual packages.	$\geq 90\%$ line
Integration Tests	test/integration/	Verify full parse scenarios.	Key workflows
SPARK Tests	test/spark/	Formal verification.	100% proof

3.2. Testing Philosophy

- Test behavior, not implementation.
- Each test validates one logical concept.
- Use descriptive test names.
- No mocking — test against real Ada.Command_Line behavior.
- SPARK verification complements dynamic testing.

3.3. Coverage Requirements

Metric	Target
Line Coverage	$\geq 90\%$
Branch Coverage	$\geq 85\%$
MC/DC Coverage	$\geq 80\%$ (SPARK packages)

4. Test Organization

4.1. Directory Structure

```
test/
├─ common/           # Shared test framework
│   └─ test_framework.ads
│   └─ test_framework.adb
```

```

├─ unit/                                # Unit tests
│   ├── unit_tests.gpr
│   ├── unit_runner.adb
│   ├── test_flag.adb
│   ├── test_option.adb
│   └── test_positional.adb
├─ integration/                        # Integration tests
│   ├── integration_tests.gpr
│   └── test_full_parse.adb
└─ spark/                             # SPARK instantiation tests
    └── spark_instantiations.ads

```

4.2. Build/Test Projects

Project	Purpose
test/unit/unit_tests.gpr	Builds unit test executables.
test/integration/integration_tests.gpr	Builds integration test executables.

5. Test Execution

5.1. Running All Tests

`make test-all`

5.2. Running Specific Suites

`make test-unit`

`make test-integration`

`make spark-prove`

6. Test Details

6.1. Unit Tests

6.1.1. Flag Tests

Test Case	Description	Expected Result
TC-FLAG-001	Flag not present.	Is_Set returns False, Count = 0.
TC-FLAG-002	Short flag present (-v).	Is_Set returns True, Count = 1.
TC-FLAG-003	Long flag present (--verbose).	Is_Set returns True, Count = 1.
TC-FLAG-004	Multiple short flags (-vvv).	Count = 3.
TC-FLAG-005	Combined short flags (-vh).	Both flags set.

6.1.2. Option Tests

Test Case	Description	Expected Result
TC-OPT-001	Option not present.	Has_Value False, Value returns None.
TC-OPT-002	Short option with space (-o file).	Value = "file".
TC-OPT-003	Long option with equals (--output=file).	Value = "file".
TC-OPT-004	Long option with space (--output file).	Value = "file".
TC-OPT-005	Option with "or" default.	Returns default when None.

Test Case	Description	Expected Result
TC-OPT-006	Required option missing.	Parse returns Error.

6.1.3. Positional Tests

Test Case	Description	Expected Result
TC-POS-001	No positional arguments.	Values is empty.
TC-POS-002	Single positional argument.	Values contains 1 element.
TC-POS-003	Multiple positional arguments.	Values contains all elements.
TC-POS-004	Positional after flags.	Correctly separated.
TC-POS-005	Positional with - - separator.	Arguments after - - are positional.

6.1.4. Parse Error Tests

Test Case	Description	Expected Result
TC-ERR-001	Unknown switch.	Error with Unknown_Switch kind.
TC-ERR-002	Missing option value.	Error with Missing_Value kind.
TC-ERR-003	Valid parse.	Ok(Unit) returned.
TC-ERR-004	Help flag (- -help).	Error with Help_Requested kind.

6.1.5. Help Generation Tests

Test Case	Description	Expected Result
TC-HELP-001	Help contains application name.	Name appears in output.
TC-HELP-002	Help contains description.	Description appears.
TC-HELP-003	Help lists all flags.	All flags shown with help text.
TC-HELP-004	Help lists all options.	All options shown with value names.

6.2. Integration Tests

6.2.1. Full Parse Scenarios

Test Case	Command Line	Expected Result
TC-INT-001	app	No flags set, no positionals.
TC-INT-002	app -v file.txt	Verbose set, 1 positional.
TC-INT-003	app --output=out.txt file1 file2	Output = "out.txt", 2 positionals.
TC-INT-004	app -vh --output file.txt input.ada	Multiple flags, option, positional.
TC-INT-005	app --help	Help_Requested error (graceful exit).
TC-INT-006	app --unknown	Unknown_Switch error.

6.2.2. Edge Cases

Test Case	Description	Expected Result
TC-EDGE-001	Empty string argument.	Handled gracefully.
TC-EDGE-002	Argument with spaces (quoted).	Preserved correctly.
TC-EDGE-003	Very long argument.	Within bounds or error.

Test Case	Description	Expected Result
TC-EDGE-004	Unicode in arguments.	Handled correctly.
TC-EDGE-005	Argument starting with - after --.	Treated as positional.

6.3. SPARK Verification

6.3.1. Packages Under SPARK

Package	SPARK_Mode	Proof Level
Clara	On	Full.
Clara.Types	On	Full.
Clara.Errors	On	Full.
Clara.Version	On	Full.
Clara.Application (core)	On	Full.

6.3.2. Proof Objectives

Objective	Verification
No runtime errors	GNATprove level 2.
No uninitialized reads	Flow analysis.
Bounded string access	Range checks proven.
Array bounds	Index checks proven.

7. Traceability

7.1. Requirements to Tests

Requirement	Test Coverage
REQ-APP-001..004	TC-INT-001..006
REQ-FLAG-001..006	TC-FLAG-001..005
REQ-OPT-001..007	TC-OPT-001..006
REQ-POS-001..004	TC-POS-001..005
REQ-ERR-001..004	TC-ERR-001..004
REQ-HELP-001..005	TC-HELP-001..004

8. Test Maintenance

8.1. When to Update

- Add tests when new argument types or features are introduced.
- Add regression tests when defects are fixed.
- Update tests when parsing behavior changes.
- Revise SPARK expectations when verification scope evolves.

8.2. Quality Guidelines

- Each public operation should have at least one test.
- Test both valid and invalid inputs.
- Test edge cases (empty, boundary values, long strings).

- Keep tests independent (no shared mutable state).

8.3. Continuous Integration

Tests run automatically on pull request creation, push to main, and release tag creation.

9. Appendices

9.1. Change History

Version	Date	Author	Changes
0.1.0	2025-12-19	Michael Gardner	Initial test strategy. Migrate to Typst format.